

Langages de programmation d'applications Web

Langage de développement d'une application web

- Script côté **client**

- le code qui s'exécute dans un navigateur web et détermine les clients (visiteurs) verront quand ils arrivent sur votre site web.

- Script côté **serveur**

- Le code qui s'exécute sur un serveur web et qui alimente la mécanique derrière les coulisses du fonctionnement d'une application web.

- Technologies des **Bases de Données**

- Contribuant à rendre une application web dynamique

Plusieurs langages de programmation

- Script côté **client (front)**

- HTML / CSS

- JavaScript

- Flash

- JQuery

- Microsoft Silverlight

- Angular

- React

- Vue.JS

- Script côté **serveur (back)**

- PHP

- C

- Java

- Ruby

- Python

- JavaScript (NodeJS)

- Express JS

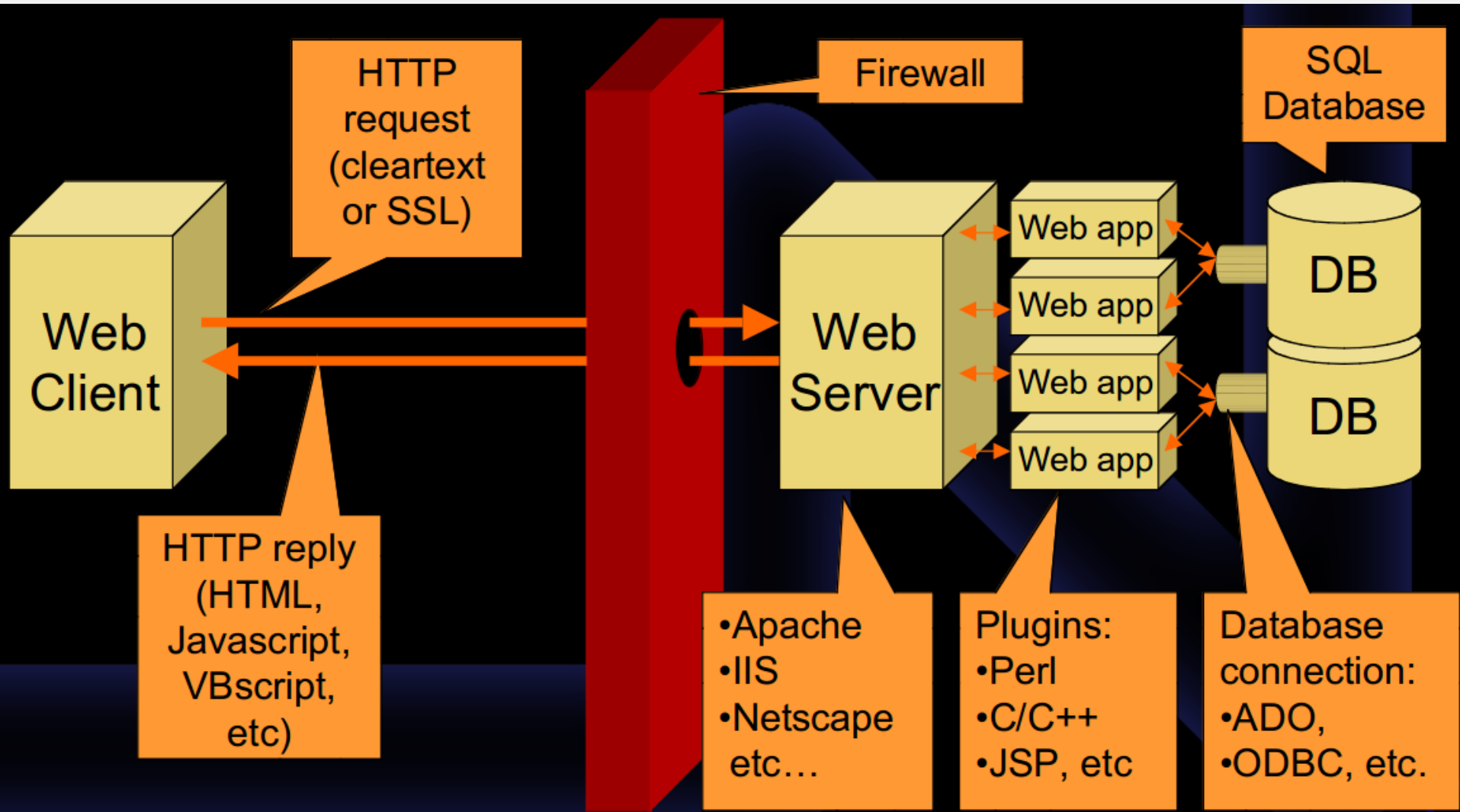
- Django, Flask

- Ruby on Rails

- Spring Boot

- Laravel, Symfony

Configuration typique d'une application web



Langage HTML (HyperText Mark-Up Language)

Langage HTML (HyperText Mark-Up Language)

- Langage de description, de “marquage”
 - Composé des balises doubles (ouvrant et fermant) ou simples
 - Les balises ne sont pas sensibles à la casse
 - Formaliser l'écriture d'un document avec des balises de formatage
 - Lecture de documents sur **Internet** à partir de machines différentes, grâce au protocole **HTTP**
- Balises doubles :
 - `<TITLE>Cours de développement <Web</TITLE>`
 - `<H1 ALIGN=JUSTIFY>Texte justifié</H1>`
 - `<HEAD>...</HEAD>`
- Balise simple
 - `<BR/>` ou encore
 - `<IMG scr= "images/Logo_Uni.png" alt="Logo Uni Ndere"/>`

Langage HTML

```
<html>
<head>
  <title>Durée du cours</title>
<metadata keywords="cours, université, Ngaoundéré">
</head>
<body>
  Saisir un cours, son enseignant et sa durée:
  
  <h2>Saisir un cours, son enseignant et sa durée: </h2>
  <form action="formulaire.php" method="post">
    Cours: <input type="text" size=20 name="cours"><br/>
    Enseignant:<input type="text" size=20 name="enseignant">
<br/>
    Durée: <select name="duree">
      <option value="2">2 heures</option >
      <option value ="3">3 heures</option >
      <option value ="4">4 heures</option >
    </select >
    <input type="submit" value="Envoyer">
  </form>
</body></html>
```

Les formulaires

- Formulaires réelle interface entre l'utilisateur et le web
- Transmission des données sous la forme :
nom1=val1&nom2=val2&...
- Méthodes **POST** et **GET** livrent les données au serveur ou script
 - GET place les variables et leurs valeurs le dans l'URL
 - POST les place dans le corps du protocole **http**

Les formulaires

- Le champ **INPUT** dont l'attribut **TYPE** prend l'une des valeurs
 - **TEXT** pour les chaînes de caractères
 - **HIDDEN** pour les champs cachés
 - **CHECKBOX** pour un choix multiple
 - **RADIO** pour un choix exclusif
 - **SUBMIT** pour déclencher l'action
 - **FILE** pour transmettre un fichier

Les formulaires

- **Exemples**

```
<input type="radio" name="drink" value="water">Eau<br>
<input type="radio" name="drink" value="beer">Bière<br>
<input type="radio" name="drink" value="wine"
checked>Vin<br>
```

- **Le champ TEXTAREA pour le texte de plusieurs lignes :**

```
<textarea name='champ' cols='30' rows='4'>
    Entrez votre texte de plusieurs ligne.
    Entrez votre texte de plusieurs ligne.
</textarea>
```

- **SELECT pour champs déroulant**

```
<select name='club'>
<option value ='1'>Coton Sport </option >
< option value ='2'>Canon</option >
< option value ='3'>Tiko United </option >
< option value='4'>Ngaoundéré Université </option >
</select>
```

Cascading Style Sheet (CSS)

- Feuilles de style en cascade

- Sert à la décrire la présentation des documents HTML et XML
- Séparation des styles de présentation de la structure du document
- Standards définissant CSS sont publiés W3C

- Structure de base

```
sélecteur {  
    propriété: valeur;  
}
```

Le sélecteur: baslise HTML: **body** ; **p** ; **img**, etc.,
l'identifiant (**id**) ou la classe (**class**)

La propriété: **font** ; **background** ; **margin** ; etc.

La valeur: les caractéristiques de la propriété

- Exemple

```
p {  
    font-size: 100%;  
    font-family: Helvetica, sans-serif;  
    color:#FF0000  
}
```

Intégration du CSS dans un document HTML

```
....  
<title>Durée du cours</title>  
<style type="text/css">  
    .myP { font-size: 16pt; font-family: Helvetica, sans-  
        serif; color:#FF0000 }  
</style>  
</head>  
<body>  
<p class=myP>Saisir un cours, son enseignant et sa  
durée</p>  
....
```

Feuille de style interne

```
....  
<title>Durée du cours</title>  
<link rel= "stylesheet" href= "style.css"  
type="text/css">  
</head>  
<body>  
Saisir un cours, son enseignant et sa durée:  
....
```

Feuille de style externe

```
<p style="text-align:center; color:red">  
    lorem ipsum  
</p>
```

Feuille de style inline

HTML5 et JavaScript

- 5^é version de la technologie HTML
- Mais
 - ensemble : **HTML5**, **CSS3** et **JavaScript**
- Des nouveaux types



- | | | |
|----------|----------|------------------|
| • search | • number | • time |
| • email | • Range | • datetime |
| • Url | • date | • datetime-local |
| • tel | • month | • color |
| | • week | |

Ref : <https://developer.mozilla.org/fr/docs/Web/Guide/HTML/HTML5>

Tests et exemples

JavaScript et Utilité

- Rendre le site web réactif aux interactions des utilisateurs avec des éléments d'un formulaire (input, text area, button, radio button, checkbox, selection list) and hypertext links
- Distribuer des petites collections de données et fournir une interface confortable
- Contrôler la navigation dans les frames, les plug-ins
- Traiter les données avant de les envoyer au serveur
- Changer dynamiquement le contenu les styles en réponse aux interactions des utilisateurs

JavaScript

- JavaScript en interne dans une source HTML

```
<html>
  <head>
    <title>Ma page</title>
    <script type="text/javascript" charset="utf-8">
      function sayHello() {
        document.getElementById('foo').innerHTML = 'Hi there!';
      }
    </script>
  </head>
  <body>
    <h1 id="foo" onclick = "sayHello()">Clique moi!</h1>
  </body>
</html>
```

JavaScript

- JavaScript en externe dans une source HTML

```
var foods = ['Mangue', 'Banane', 'Orange'];  
for (var i=0; i<foods.length; i++) {  
    if (foods[i] == 'mangue') {  
        alert(foods[i] + ' mon fruit favori!');  
    } else {  
        alert(foods[i] + ' est ok.');    }  
}
```

myfile.js



```
<html> ...  
    <script type="text/javascript"  
src="./mesFichiersJS/myfile.js"></scri  
pt>  
    <body>  
...  
    </body>  
</html>
```

Travaux pratiques individuels

Créer une seule page html contenant tous types de champ HTML5, un fichier CSS3 pour le design et trois fonctions JavaScript

Guide:

- 1ère fonction JS calcule la taxe à retenir sur un prix d'un article entré dans un champs (17%).
- 2e fonction JS affiche la date du jour.
- 3e fonction JS vérifie l'existence d'un élément dans un Array des nombres entiers et renvoie une valeur booléenne true ou false selon le cas.

Introduction au langage PHP

PHP Scripts

- langage interprété indépendant de la plate-forme d'exécution
s'exécute sur le serveur
- les instructions sont intégrées au code source du document

HTML

- permet de générer des pages HTML

```
<html> <head>
</head>
<body>
<H1> Voici l'email de l'administrateur du serveur </H1>
<br>
<?php
    echo $_SERVER[ 'SERVER_ADMIN' ]
?> </body> </html>
```

PHP Scripts

- L'interpréteur du code PHP se trouve au serveur HTTP
- Le serveur lit les instructions PHP intégrés à la page HTML (entre `< ?php ... ?>`), les interprète et les remplace par le résultat de leur exécution
- La page HTML générée par le serveur est envoyée au client
- Le navigateur au niveau client affiche la page HTML
- Un fichier PHP (.php) peut contenir
 - du code HTML
 - du code PHP
 - du code JavaScript

Intégrer du PHP dans une page HTML

- La façon la plus simple :

```
<?php echo "Salut" ; ?>
```

- Autre méthode :

```
<script language="php"> echo "Salut" ; </script>
```

- On peut trouver aussi

```
<? echo "Salut" ; ?>
```

- mais pose un problème de compatibilité avec XML

Variables en PHP

- Affectation par valeur by value

```
$foo = "Bob"; $bar = $foo;
```

- Affectation par référence

```
$bar = &$foo;
```

- Des variables préaffectées, server et env vars

- Exemples, eg. `PHP_SELF`, `HTTP_GET_VARS`,

Opérateurs en PHP

- Arithmétique (+, -, *, /, %) et String (.)

- Affectation (=) et affectation combinée

```
$a = 3;
```

```
$a += 5; // sets $a to 8;
```

```
$b = "Hello ";
```

```
$b .= "There!"; // sets $b to "Hello There!";
```

- Bit (&, |, ^, ~, <<, >>)

- `$a ^ $b` (Xor: Bits that are set in \$a or \$b but not both are set.)

- `~ $a` (Not: Bits that are set in \$a are not set, and vice versa.)

- Comparaison (==, !=, <, >, <=, >=)

Opérateurs en PHP

• Logique

`$a and $b` **True** si `$a` et `$b` sont vrais.

`$a or $b` **True** si `$a` ou `$b` est vrai.

`$a xor $b` **True** si `$a` ou `$b` est vrai, mais pas les deux

`! $a` **True** si `$a` n'est pas vrai.

`$a && $b` **True** si `$a` et `$b` sont true.

`$a || $b` **True** si `$a` ou `$b` est vrai.

• Incrémenter / Décrémenter

`++$a` (Incrémente d'un et retourne `$a`)

`$a++` (Retourne `$a` et incrémente `$a` d'un)

`--$a` (Décrémente `$a` d'un et retourne `$a`)

`$a--` (Retourne `$a` et décrémente `$a` d'un)

Structures de contrôle

- Plusieurs possibilités

- `if, else, elseif`
- `while, do-while`
- `for, foreach`
- `break, continue, switch`
- `require, include, require_once, include_once`
(pour inclure un fichier externe)

Switch et if

- Deux possibilités de faire la même chose

```
if ($i == 0) {  
    echo "i equals 0";  
} elseif ($i == 1) {  
    echo "i equals 1";  
} elseif ($i == 2) {  
    echo "i equals 2";  
}
```

```
switch ($i) {  
case 0:  
    echo "i equals 0";  
    break;  
case 1:  
    echo "i equals 1";  
    break;  
case 2:  
    echo "i equals 2";  
    break;  
}
```

Traverser des Arrays

- Utiliser la boucle `foreach` pour traverser un champs

```
$colors = array('red', 'blue', 'green', 'yellow');  
  
foreach ($colors as $color) {  
    echo "<p>Do you like $color?</p>\n";  
}
```

Fonctions intéressantes

- `str_replace()`
- `trim()`, `ltrim()`, `rtrim()`
- `implode()`, `explode()`
- `addslashes()`, `stripslashes()`
- `htmlentities()`, `html_entity_decode()`, `htmlspecialchars()`
- `striptags()`

Remove the backslash:

```
<?php
echo stripslashes("Who\'s Peter Griffin?");
?>
```

Break a string into an array:

```
<?php
$str = "Hello world. It's a beautiful day.";
print_r (explode(" ", $str));
?>
```

Strip the string from HTML tags:

```
<?php
echo strip_tags("Hello <b>world!</b>");
?>
```

```
<!DOCTYPE html>
<html>
<body>

<?php
$str = addslashes('What does "yolo" mean?');
echo($str);
?>

</body>
</html>
```

Replace the characters "WORLD" (case-insensitive) in the string "Hello world!" with "Peter":

```
<?php
echo str_ireplace("WORLD", "Peter", "Hello world!");
?>
```

Fonctions intéressantes

- `str_replace()`
- `trim()`, `ltrim()`, `rtrim()`
- `implode()`, `explode()`
- `addslashes()`, `stripslashes()`
- `htmlentities()`, `html_entity_decode()`,
`htmlspecialchars()`
- `striptags()`

• Créer un champs

```
$sauce = "sauce1 sauce2 sauce3 sauce4 sauce5 sauce6";  
$sauces = explode(" ", $sauce);
```

Connexion à une base de données

```
mysql_connect("localhost", "root", "password");
```

```
mysql_select_db("ingaweb2016"); //sélectionner une BD
```

Une requête SQL

```
$result=mysql_query("SELECT * from userdetail WHERE name  
LIKE ('%test')");
```

```
//vérifier si nous avons obtenu quelque chose
```

```
if (mysql_num_rows($result) == 0) {  
    echo "Aucun résultat!";  
    exit;  
}
```

Connexion à une base de données

```
while($row=mysql_fetch_assoc($result)) {  
    echo $row['...'];  
    echo $row['...'];  
}
```

- Exemple de la fonction d'autocomplete -

Travaux pratiques individuels

**Créer son propre profile avec ses
amis et préférences**

Introduction à Flask (Python)

Prérequis

- Connaissance de base Python (2 ou **3 de préférence**)
- Installation de Python et pip
- Installation Flask

Environnement et configuration

- Créer un nouveau projet (répertoire + env. virtuel Flask)

```
pip install Flask
```

- Créer 1 fichier Python (e.g. '**app.py**') et importer la classe '**Flask**'

```
from flask import Flask  
app = Flask(__name__)
```

Création de routes et exécution

- En fonction des URLs souhaitées

```
@app.route('/') ## Chemin d'accès de la route
def index():
    return 'Bienvenue sur notre application web Python'

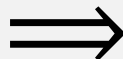
@app.route('/about')
def about(): ## nom de la route
    return 'À propos de nous'
```

- Exécution de l'application

```
if __name__ == '__main__':
    app.run(debug=True)
```

- Lancer le serveur Flask

```
python app.py
```



Accédez à l'URL :

<http://localhost:5000/>

eXtensible Markup Language (XML)

- Composants et syntaxe d'un document XML bien formé
 - Déclaration de version du langage
 - Toute balise ouverte doit être fermée
 - Les enchaînements forment un arbre strict
 - Les chevauchements sont interdits
 - Les valeurs d'attribut sont entre guillemets
 - Un document XML n'a qu'un seul élément racine.
- Document type Definition (DTD) valide le document XML
- Schéma XML pour valider le document XML

XML, son utilité

- Description des données
- Echange des données
- Echange de ressources entre deux systèmes hétérogènes
- Exemples d'usage
 - LibreOffice, OpenOffice.org, Microsoft Word
 - SQL, la plupart des SGBD proposent un export XML

XML, un exemple

- Entité **livre** et ses **attributs**

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<livre>
  <titre>Ingénierie des applications web</titre>
  <auteur>Souley</auteur>
  <auteur>Boulmo Richard</auteur>
  <edition>Rabiatou</edition>
  <chapitre titre="Introduction">
    <texte>Texte pour le chapitre d'introduction
  </texte>
    <image file="logo_uni.png"/>
  </chapitre>
  <chapitre titre="References" type="bibliographie">
    <texte>Texte pour le chapitre références
  </texte>
  </chapitre>
</livre>
```

XML vs HTML

Différence entre **XML et HTML**

XML	HTML
XML signifie eXtensible Markup Language	HTML signifie HyperText Markup Language
Le XML est utilisé principalement pour l'échange de données entre deux partenaires	Le HTML est utilisé pour la structure des pages web
Syntaxe libre pour les balises	Syntaxe stricte Chaque balise à un rôle
Aucune erreur permise. Fermeture des balises obligatoire	Erreurs minimales possibles. Fermeture non indispensable
Extension de fichier en .xml	Extension de fichier en .html ou .htm

Document Type Definition (DTD)

- Il permet de définir précisément la structure d'un document.
- Définissent des contraintes que doit respecter un document pour être valide.
- Ces contraintes spécifient les éléments qui peuvent apparaître dans le contenu d'un élément, l'ordre éventuel de ces éléments et la présence de texte brut.
- Validation de document XML décrivant l'entité **livre**
- Définition de Type de Document: Document Type Definition (DTD)
 - Décrire un document SGML (**Standard Generalized Markup Language**) ou XML
 - Grammaire de classe de documents
 - Description de la position des termes les uns par rapport aux autres: **grammaire**
 - Généralisation d'un domaine particulier: **classe**

Document Type Definition (DTD)

- Un DTD pour valider le document XML décrivant **livre**

```
<!DOCTYPE livre
[
<!ELEMENT livre (titre, auteur+, edition?, chapitre+)>
<!ELEMENT titre (#PCDATA)>
<!ELEMENT auteur (#PCDATA)>
<!ELEMENT edition (#PCDATA)>
<!ELEMENT chapitre (texte+, image)>
<!ELEMENT texte (#PCDATA)>
<!ELEMENT image EMPTY>
]>
```